PRD-06

Production Engineering & Scalability

Version: 1.0

Objective

Transform the current backend into a production-ready, scalable, secure, observable, and maintainable system capable of supporting high traffic and future growth.

This PRD focuses on infrastructure, performance, caching, background processing, monitoring, testing, deployment, and operational readiness rather than adding end-user features.

Scope

This PRD includes:

- Redis Caching
- Background Job Queue
- Email Queue
- Scheduled Jobs
- WebSocket Gateway
- API Versioning
- Rate Limiting Improvements
- Monitoring
- Logging Improvements
- Docker
- CI/CD
- Testing
- Security Hardening
- Performance Optimization
- Database Optimization
- Backup Strategy
- Health Monitoring

---

1. Redis Integration

Integrate Redis for caching and ephemeral data.

Use cases:

- Frequently accessed roadmaps
- Categories
- Learning resources
- Dashboard summaries
- Notifications count
- Search suggestions
- Rate limiting
- Session cache

Requirements:

- Centralized cache service
- Configurable TTL
- Cache invalidation on updates
- Graceful fallback if Redis is unavailable

---

2. Background Job Queue

Integrate BullMQ (or equivalent) backed by Redis.

Job types:

- Email sending
- Notification processing
- Resume processing
- Analytics aggregation
- Cleanup tasks
- Scheduled reminders

Requirements:

- Retry support
- Exponential backoff
- Dead-letter handling
- Job status monitoring

---

3. Email Service

Move email sending to background jobs.

Supported emails:

- Email verification
- Password reset
- Team invitation
- Event registration
- Placement reminders
- Weekly learning summary

---

4. Scheduled Jobs

Implement cron-based jobs for:

- Expire invitations
- Delete expired OTPs
- Archive expired events
- Refresh analytics
- Generate daily coding challenge
- Weekly learning reports
- Cleanup stale notifications

---

5. WebSocket Gateway

Implement real-time communication.

Support:

- Live notifications
- Team activity updates
- Invitation updates
- Task status updates
- Dashboard refresh events

Design the gateway so future chat and collaborative editing can be added without architectural changes.

---

6. API Versioning

Adopt URL-based versioning:

/api/v1/...

Keep routing ready for future `/api/v2` endpoints.

---

7. Rate Limiting Enhancements

Introduce endpoint-specific limits.

Examples:

- Authentication
- File uploads
- Search
- AI endpoints (future)
- Notifications

Store counters in Redis.

---

8. Monitoring & Observability

Expose metrics endpoint.

Track:

- Request count
- Response times
- Error rates
- Database query duration
- Cache hit/miss ratio
- Queue sizes
- Active users
- Memory usage

Prepare integration with Prometheus/Grafana.

---

9. Logging Improvements

Enhance logging with:

- Correlation IDs

- Structured JSON logs
- Request tracing
- Queue logs
- Cache logs
- Database slow query logs

Support different log levels per environment.

---

10. Docker

Provide:

- Dockerfile
- docker-compose.yml
- Separate services:
    - API
    - Redis
    - PostgreSQL (development)
    - Queue worker

Support one-command local startup.

---

11. CI/CD

Configure GitHub Actions.

Pipeline:

- Install dependencies
- Lint
- Type check
- Run tests
- Build
- Generate Prisma client
- Verify migrations
- Publish artifacts (future deployment hook)

---

12. Testing

Introduce:

Unit Tests

Cover:

- Services
- Utilities
- Validators

Integration Tests

Cover:

- Authentication
- Learning
- Coding
- Projects
- Placement

API Tests

Verify:

- Status codes
- Authorization
- Validation
- Error handling

Aim for at least **80% coverage** on business logic.

---

13. Security Hardening

Implement:

- CSP headers
- Secure cookies
- Input sanitization
- Refresh token rotation
- Secret validation at startup
- Dependency vulnerability scanning
- File upload validation (type and size)
- Audit logging for admin actions

---

14. Database Optimization

Review:

- Index usage
- Query plans
- Connection pooling
- N+1 queries
- Composite indexes
- Batch operations

Add database health checks.

---

15. Backup & Recovery

Document and automate:

- Nightly backups
- Restore process
- Migration rollback strategy

---

16. Health Endpoints

Provide:

GET /api/v1/health
GET /api/v1/health/database
GET /api/v1/health/cache
GET /api/v1/health/queue

---

17. Configuration Management

Centralize application configuration.

Validate all required environment variables at startup.

Support:

- Development
- Staging
- Production

---

18. Performance Targets

- API median response time < 200 ms for cached endpoints.
- P95 response time < 500 ms under normal load.
- Background jobs processed asynchronously without blocking requests.
- Graceful degradation if Redis or queue services are temporarily unavailable.

Acceptance Criteria

PRD-06 is complete when:

- Redis is integrated and used for caching and rate limiting.
- Background job processing is operational.
- Email sending is asynchronous.
- Scheduled jobs execute reliably.
- WebSocket gateway broadcasts supported events.
- API versioning is implemented.
- Monitoring endpoints expose application metrics.
- Structured logging with correlation IDs is in place.
- Docker configuration supports local development.
- CI/CD pipeline passes automatically.
- Test suite covers core business logic and API workflows.
- Security hardening measures are implemented.
- Health checks report application, database, cache, and queue status.
- Configuration validation prevents startup with missing critical settings.

After PRD-06

At this point, I **would stop backend development** unless new product requirements emerge.

The backend would be in a strong position to support frontend development and iterative improvements.

The next major focus should be:

1. **Frontend integration** with the completed APIs.
2. End-to-end testing across the full application.
3. UX refinement.
4. Performance tuning based on real usage.
5. Deployment to a staging environment before public release.

This shifts the project from "building features" to "delivering a polished product," which is where the greatest value comes once the core platform is complete.